

AI Critique

Gemini was useful for turning the assignment constraints into an initial workload model, but its first answer mixed source facts, common performance-testing patterns, and unsupported predictions. It claimed that `SQLITE_BUSY` would be the primary bottleneck and that Node.js would saturate one core first. The implementation actually opens one shared `sqlite3` connection, so queuing was at least as plausible; the measured Stress run produced no lock errors and instead showed a capacity knee: throughput changed only from 324.06 to 326.86 samples/s between 30 and 40 users while p95 rose from 74 to 109 ms. Gemini also inherited my incorrect statement that duplicate emails fail. Source review and two real requests proved the opposite: both returned HTTP 200 with different IDs. Its data-isolation recommendation-50,000 CSV rows plus replacing an open database file-was unnecessary and unsafe. A UUID email and controlled backend restart were more defensible.

The revised answer improved after a focused corrective prompt, yet still suggested substring checks for the GET response and a questionable listener choice. Substrings could match an ID and email from different objects, so the final JMeter assertion parses JSON and requires both fields on the same user object. These failures occurred because the model optimized for a plausible generic design before seeing code and raw measurements; confident wording hid that uncertainty. I learned to separate facts, hypotheses, and measured values in every prompt, then test each AI claim against source, raw JTL rows, and repeatable API evidence. The productive collaboration pattern was not "ask once and accept." It was propose, inspect, challenge with concrete counter-evidence, implement narrowly, and recalculate metrics independently before using AI recommendations.

Word count: **269** (body only, whitespace-separated count).